Treniņu plāna un rezultātu sistēma

Pauls Dubults

1. Problēmas izpēte un analīze **4**
1.1. Problēmas apraksts 4
1.2. Projekta mērķis 4
1.3. Mērķauditorija 4
1.4. Lietotāja vajadzību izpēte 5
Novērošana 5
Salīdzināšana ar līdzīgiem risinājumiem 5
Pašanalīze un vajadzību formulēšana 5
1.5. Izpētes secinājumi 5
2. Programmatūras prasību specifikācija **5**
2.1. Programmatūras apraksts 5
2.2. Funkcionālās prasības 6
2.3. Nefunkcionālās prasības 6
2.4. Lietotājstāsti 6
TPS_L1 6
TPS_L2 6
TPS_L3 6
TPS_A1 6
TPS_A2 7
3. Programmatūras projektējums **7**
3.1. Izvēlētās tehnoloģijas un to pamatojums 7
3.2. Programmatūras struktūrskice 7
3.3. Datubāzes ER modelis 7
3.4. Galveno funkciju apraksts 8
Reģistrācija un pieslēgšanās 8
Treniņu plāna apskate 8
Rezultātu pievienošana 8
Rezultātu apskate 8
Administratora funkcijas 8
3.5. Datu struktūras izmantošana 8
4. Programmatūras izstrādes plāns **8**
4.1. Programmatūras izstrādes modeļu salīdzinājums 8
Ūdenskrituma modelis 8
Agile jeb prototipēšanas pieeja 9
4.2. Izvēlētais modelis 9
4.3. Vienkāršots izstrādes plāns 9
5. Atkļūdošanas un akceptestēšanas pārskats **9**
5.1. Atkļūdošanas process 9
5.2. Biežāk atrastās kļūdas 10
5.3. Akceptestēšanas pārskats 10
AT-01 10
AT-02 10
AT-03 10
AT-04 10

AT-05 10
AT-06 10
AT-07 10
5.4. Secinājumi par testēšanu 11
6. Lietotāja ceļvedis 11
6.1. Sistēmas uzstādīšana 11
6.2. Pieslēgšanās sistēmai 11
6.3. Treniņu plāna apskate 12
6.4. Rezultātu pievienošana 12
6.5. Rezultātu apskate 12
6.6. Administratora darbības 12
7. Pielikums 12
7.1. Programmatūras projekta publiskā GitHub repozitorija adrese 12
7.2. Projekta failu un mapju struktūras apraksts 12

1. Problēmas izpēte un analīze

1.1. Problēmas apraksts

Mūsdienās daudzi cilvēki savus treniņu plānus un rezultātus uzglabā dažādos veidos — piezīmju lietotnēs, papīra pierakstos, telefonā vai vienkārši atceroties. Šāda pieeja nav ērta, jo informācija bieži kļūst nesakārtota, grūti pārskatāma un viegli pazaudējama. Lietotājam ir sarežģīti sekot līdzi progresam, salīdzināt iepriekšējos rezultātus un saprast, kā mainās viņa sniegums laika gaitā.

Papildus tam, ja treniņu plānus veido cita persona, piemēram, treneris vai administrators, tad bez vienotas sistēmas ir grūti pārvaldīt vingrinājumus, piešķirt plānus konkrētiem lietotājiem un nodrošināt datu pārskatāmību. Tāpēc radās nepieciešamība izveidot vienkāršu tīmekļa sistēmu, kas ļautu vienuviet pārvaldīt treniņu plānus un ievadīt rezultātus.

1.2. Projekta mērķis

Projekta mērķis ir izveidot tīmekļa lietotni, kas ļauj lietotājam apskatīt sev piešķirto treniņu plānu, ievadīt savus treniņu rezultātus un sekot līdzi progresam, savukārt administratoram pievienot vingrinājumus un pārvaldīt lietotāju treniņu plānus.

1.3. Mērķauditorija

Šīs programmatūras mērķauditorija ir:

- cilvēki, kuri regulāri trenējas sporta zālē;

- jaunieši, kuri vēlas sekot līdzi savam progresam;
- lietotāji, kuriem nepieciešams vienkāršs un saprotams treniņu pierakstu rīks;
- administrators vai treneris, kurš pārvalda vingrinājumus un plānus.

1.4. Lietotāja vajadzību izpēte

Lai noteiktu, kādai funkcionalitātei sistēmā jābūt, tika veikta vienkāršota lietotāju vajadzību izpēte. Tika izmantotas šādas metodes:

Novērošana

Tika analizēts, kā praksē cilvēki pieraksta treniņu rezultātus. Bieži tika izmantotas telefona piezīmes, nejauši pieraksti vai vairākas atsevišķas lietotnes, kas padara informāciju grūti pārskatāmu.

Salīdzināšana ar līdzīgiem risinājumiem

Tika apskatītas vairākas esošās treniņu uzskaites lietotnes un secināts, ka daudzas no tām ir pārāk sarežģītas, pārslogotas ar funkcijām vai nav piemērotas vienkāršai skolas projekta realizācijai.

Pašanalīze un vajadzību formulēšana

Tika izvērtētas galvenās funkcijas, kas šādā sistēmā būtu praktiski noderīgas. Svarīgākās no tām bija:

- redzēt savu treniņu plānu;
- ievadīt rezultātus;
- saglabāt datus datubāzē;
- nodrošināt, ka lietotājs redz tikai savus datus;
- dot administratoram iespēju pārvaldīt sistēmas saturu.

1.5. Izpētes secinājumi

Pēc vajadzību izpētes tika secināts, ka lietotājiem svarīgākais ir vienkāršība, pārskatāmība un drošība. Sistēmai nav jābūt ļoti sarežģītai, bet tai ir jāspēj izpildīt pamatfunkcijas: lietotāja autorizācija, treniņu plāna apskate, rezultātu pievienošana un administratora pārvaldība. Tika nolemts veidot tīmekļa sistēmu, jo to ir iespējams ērti lietot pārlūkprogrammā bez papildu instalācijas.

2. Programmatūras prasību specifikācija

2.1. Programmatūras apraksts

Izstrādātā programmatūra ir tīmekļa lietotne, kurā lietotāji var pieslēgties sistēmai, apskatīt savu treniņu plānu un pievienot treniņu rezultātus. Sistēmā ir arī administratora loma, kas ļauj pievienot vingrinājumus un pārvaldīt treniņu plānus.

2.2. Funkcionālās prasības

Programmatūrai ir jānodrošina šādas funkcijas:

1. Lietotājam jāspēj reģistrēties sistēmā.
2. Lietotājam jāspēj pieslēgties sistēmai.
3. Lietotājam jāspēj iziet no sistēmas.
4. Lietotājam jāspēj apskatīt savu treniņu plānu.
5. Lietotājam jāspēj pievienot sava treniņa rezultātu.
6. Lietotājam jāspēj apskatīt savus iepriekš ievadītos rezultātus.
7. Administratoram jāspēj pievienot jaunus vingrinājumus.
8. Administratoram jāspēj apskatīt lietotājus.
9. Administratoram jāspēj piešķirt treniņu plānu lietotājam.
10. Sistēmai jānodala piekļuve pēc lietotāja lomas.
11. Sistēmai jāglabā dati datubāzē.
12. Sistēmai jāattēlo lietotāja progresu grafikā.

2.3. Nefunkcionālās prasības

1. Sistēmai jābūt lietojamai tīmekļa pārlūkprogrammā.
2. Sistēmas saskarnei jābūt vienkāršai un saprotamai.
3. Sistēmai jādarbojas ar saglabātiem datiem datubāzē.
4. Paroles nedrīkst glabāt atklātā tekstā.
5. Katram lietotājam jāredz tikai savi dati.
6. Sistēmai jābūt strukturētai un viegli uzturamai.
7. Programmas kodam jābūt komentētam latviešu valodā.

2.4. Lietotājstāsti

TPS_L1

Kā lietotājs es vēlos pieslēgties sistēmai, lai varētu apskatīt savu treniņu plānu.

TPS_L2

Kā lietotājs es vēlos pievienot sava treniņa rezultātu, lai varētu sekot līdzi progresam.

TPS_L3

Kā lietotājs es vēlos redzēt savus iepriekšējos rezultātus, lai varētu salīdzināt sniegumu dažādos datumos.

TPS_A1

Kā administrators es vēlos pievienot vingrinājumus, lai sistēmā būtu pieejams pārvaldāms vingrinājumu saraksts.

TPS_A2

Kā administrators es vēlos piešķirt treniņu plānu lietotājam, lai katram lietotājam būtu savs individuāls plāns.

3. Programmatūras projektējums

3.1. Izvēlētās tehnoloģijas un to pamatojums

Projekta izstrādei tika izvēlēta programmēšanas valoda **Python**, jo tā ir saprotama, plaši izmantota un piemērota tīmekļa lietotņu izstrādei mācību projektos.

Tīmekļa lietotnes izstrādei tika izmantots **Flask** ietvars, jo tas ir viegls, vienkāršs un labi piemērots neliela un vidēja apjoma projektiem.

Datu glabāšanai tika izmantota **SQLite** datubāze, jo tā ir ērta izmantošanai lokālos projektos un neprasa atsevišķa servera uzstādīšanu.

Lietotāju autentifikācijai tika izmantota ārējā bibliotēka **Flask-Login**, kas palīdz pārvaldīt sesijas un aizsargāt lapas, kurām drīkst piekļūt tikai autorizēti lietotāji.

Lietotāja progresa attēlošanai tika izmantota **Chart.js** bibliotēka, kas ļauj attēlot datus grafikā.

3.2. Programmatūras struktūrskice

Programmatūra sastāv no vairākām galvenajām daļām:

- lietotāju autentifikācijas modulis;
- treniņu plāna modulis;
- rezultātu pievienošanas un apskates modulis;
- administratora modulis;
- datubāzes slānis;
- lietotāja saskarne ar HTML, CSS un JavaScript.

3.3. Datubāzes ER modelis

Sistēmā tiek izmantotas vairākas savstarpēji saistītas tabulas:

- **lietotaji**
- **vingrinajumi**

- **treninu_plani**
- **rezultati**

Galvenās saites:

- viens lietotājs var būt saistīts ar vairākiem treniņu plāniem;
- viens lietotājs var ievadīt vairākus rezultātus;
- viens vingrinājums var būt izmantots vairākos plānos;
- viens vingrinājums var būt saistīts ar vairākiem rezultātiem.

3.4. Galveno funkciju apraksts

Reģistrācija un pieslēgšanās

Lietotājs var izveidot kontu un pieslēgties sistēmai. Pieslēgšanās laikā tiek pārbaudīts lietotājvārds un parole.

Treniņu plāna apskate

Pēc pieslēgšanās lietotājs var apskatīt savu treniņu plānu, kas sakārtots pēc dienām.

Rezultātu pievienošana

Lietotājs var ievadīt informāciju par izpildīto vingrinājumu, piemēram, svaru, atkārtojumu skaitu, komplektu skaitu un piezīmes.

Rezultātu apskate

Lietotājs var redzēt savus iepriekš saglabātos rezultātus un progresa attēlojumu grafikā.

Administratora funkcijas

Administrators var pievienot jaunus vingrinājumus un piešķirt plānu konkrētiem lietotājiem.

3.5. Datu struktūras izmantošana

Projektā tiek izmantota vismaz viena datu struktūra — Python vārdnīca, lai sakārtotu treniņu plānu pa dienām. Tas palīdz ērtāk attēlot lietotāja plānu un loģiski sakārtot datus saskarnē.

4. Programmatūras izstrādes plāns

4.1. Programmatūras izstrādes modeļu salīdzinājums

Tika apskatīti divi programmatūras izstrādes modeļi:

Ūdenskrituma modelis

Šajā modelī izstrādes posmi notiek secīgi — sākumā prasību noteikšana, pēc tam projektēšana, programmēšana, testēšana un uzturēšana. Tas ir piemērots gadījumos, kad visas prasības jau sākumā ir pilnīgi skaidras.

Agile jeb prototipēšanas pieeja

Šajā modelī projekts tiek veidots pakāpeniski. Vispirms tiek izstrādāts vienkāršs pamata variants, pēc tam tas tiek papildināts ar jaunām funkcijām un uzlabojumiem.

4.2. Izvēlētais modelis

Šī projekta izstrādei tika izvēlēta **Agile pieeja**, jo projekta izstrādes laikā bija ērtāk pakāpeniski veidot funkcionalitāti un uzreiz pārbaudīt, vai tā darbojas pareizi. Šāda pieeja bija piemērota arī tāpēc, ka projektu izstrādāja viens cilvēks, un vairākas idejas tika precizētas izstrādes gaitā.

4.3. Vienkāršots izstrādes plāns

1. Tika izvēlēta projekta ideja.
2. Tika noteikta mērķauditorija un galvenās funkcijas.
3. Tika izstrādāts datubāzes modelis.
4. Tika izveidota Flask projekta struktūra.
5. Tika realizēta lietotāju reģistrācija un pieslēgšanās.
6. Tika realizēta lomu sistēma.
7. Tika izveidota treniņu plāna apskate.
8. Tika izveidota rezultātu ievade un apskate.
9. Tika pievienots progresa grafiks.
10. Tika izveidota administratora sadaļa.
11. Tika veikta atkļūdošana un testēšana.
12. Tika sagatavota dokumentācija un GitHub repozitorijs.

5. Atkļūdošanas un akcepttestēšanas pārskats

5.1. Atkļūdošanas process

Izstrādes laikā programma tika atkārtoti pārbaudīta pa daļām. Pēc katras jaunas funkcijas izveides tika veikta tās pārbaude, lai pārliecinātos, ka sistēma darbojas pareizi.

Atkļūdošanā tika pievērsta uzmanība:

- lietotāju pieslēgšanās loģikai;
- datu saglabāšanai datubāzē;
- lomu pārbaudei;

- kļūdu paziņojumiem pie nepareizi aizpildītām formām;
- rezultātu attēlošanai un grafika ielādei.

5.2. Biežāk atrastās kļūdas

1. Nepareiza datu ievade formās.
2. Nepareiza piekļuve lapām bez autorizācijas.
3. Lietotāja datu nepareiza filtrēšana.
4. Kļūdas, kas saistītas ar datubāzes relācijām.
5. Nepareiza grafika datu attēlošana.

5.3. Akcepttestēšanas pārskats

Tika veikti šādi akcepttesti:

AT-01

Scenārijs: lietotājs reģistrējas sistēmā.
Rezultāts: lietotāja konts tiek veiksmīgi izveidots.

AT-02

Scenārijs: lietotājs pieslēdzas sistēmai ar pareiziem datiem.
Rezultāts: lietotājs tiek novirzīts uz savu treniņu plānu.

AT-03

Scenārijs: lietotājs ievada nepareizu paroli.
Rezultāts: tiek parādīts kļūdas paziņojums.

AT-04

Scenārijs: lietotājs pievieno jaunu rezultātu.
Rezultāts: rezultāts tiek saglabāts datubāzē un parādās sarakstā.

AT-05

Scenārijs: parasts lietotājs mēģina atvērt administratora sadaļu.
Rezultāts: piekļuve tiek liegta.

AT-06

Scenārijs: administrators pievieno jaunu vingrinājumu.
Rezultāts: vingrinājums tiek saglabāts sistēmā.

AT-07

Scenārijs: administrators piešķir lietotājam treniņu plānu.
Rezultāts: lietotājam plāns parādās viņa sadaļā.

5.4. Secinājumi par testēšanu

Testēšanas laikā tika pārliecināts, ka galvenās sistēmas funkcijas darbojas pareizi. Atrastās kļūdas tika pakāpeniski novērstas, līdz sistēma sasniedza izmantojamu un demonstrējamu stāvokli.

6. Lietotāja ceļvedis

6.1. Sistēmas uzstādīšana

Lai palaistu projektu lokāli, nepieciešams:

1. Lejupielādēt projekta failus.
2. Atvērt projekta mapi terminālī.
3. Izveidot virtuālo vidi:

```
python -m venv .venv
```

4. Aktivizēt virtuālo vidi:
   Windows:

```
.venv\Scripts\activate
```

Mac / Linux:

```
source .venv/bin/activate
```

5. Uzinstalēt bibliotēkas:

```
pip install -r requirements.txt
```

6. Izveidot testa datus:

```
python seed_data.py
```

7. Palaist projektu:

```
flask --app run.py run
```

8. Atvērt pārlūkā adresi:

http://127.0.0.1:5000

6.2. Pieslēgšanās sistēmai

Lietotājs atver sistēmu pārlūkā, ievada savu lietotājvārdu un paroli, un pieslēdzas sistēmai.

6.3. Treniņu plāna apskate

Pēc pieslēgšanās lietotājs atver savu plāna sadaļu, kur redz sev piešķirtos vingrinājumus.

6.4. Rezultātu pievienošana

Lietotājs atver rezultātu pievienošanas lapu, izvēlas vingrinājumu, ievada datumu, svaru, atkārtojumus, komplektus un piezīmes, pēc tam saglabā ierakstu.

6.5. Rezultātu apskate

Lietotājs var apskatīt savu iepriekšējo rezultātu sarakstu un progresa grafiku.

6.6. Administratora darbības

Administrators var:

- pievienot jaunus vingrinājumus;
- apskatīt lietotājus;
- piešķirt plānus lietotājiem.

7. Pielikums

7.1. Programmatūras projekta publiskā GitHub repozitorija adrese

https://github.com/PaulsDubults/treninu-sistema

7.2. Projekta failu un mapju struktūras apraksts

- app/ — galvenā programmas loģika
- docs/ — dokumentācija un saistītie faili
- static/ — CSS un JavaScript faili
- templates/ — HTML veidnes
- tests/ — testēšanas faili
- run.py — projekta palaišanas fails
- seed_data.py — testa datu izveides fails

- requirements.txt — izmantoto bibliotēku saraksts
- README.md — īss projekta apraksts un palaišanas instrukcija